



Introduction to Compute Virtualization – Part 2



Foreword

- Cloud computing 1.0 focused on virtualization which today has become the foundation of cloud computing. This chapter focuses on the architecture and technologies involved in compute virtualization.



Objectives

- Upon completion of this chapter, you will:
 - Be able to describe what I/O virtualization is.
 - Understand the differences between virtualization and cloud computing.
 - Understand KVM technology.



Contents

1. Introduction to Virtualization

- CPU Virtualization
- Memory Virtualization
- **I/O Virtualization**

2. Understand the differences between virtualization and cloud computing.

3. Understand KVM technology.



I/O Virtualization

- With compute virtualization, a large number of VMs can be created on a single host, and these VMs all need to access the I/O devices of this host.
- However, I/O devices are limited. I/O device sharing among multiple VMs requires VMM. **VMM intercepts access requests from VMs to I/O devices**, simulates I/O devices using software, and responds to I/O requests. This way, multiple VMs can access I/O resources concurrently.



I/O Virtualization

- I/O virtualization can be implemented in the following methods:
 - Full virtualization,
 - Paravirtualization,
 - Hardware-assisted virtualization



I/O Virtualization

- **Emulation [Full Virtualization]:**
- VMM virtualizes I/O devices for VMs. When a VM initiates an I/O request to an I/O device, **VMM** intercepts the request sent by the VM, and **then sends the real access request to the physical device for processing.**
- No matter which type of OS is used by the VM, the OS does not need to be modified for I/O virtualization.



I/O Virtualization

- **Emulation [Full Virtualization]:**
- VMM needs to intercept I/O requests delivered by each VM in real time and emulates the request to a real I/O device. Real-time monitoring and emulation are implemented by software programs on the CPU, which **causes severe performance loss to the server.**
- Complete simulation of the hardware, for example, keyboard and mouse. Access to such hardware depends on the capture of focus by the host, and leads to poor performance in some cases.



I/O Virtualization

- **Paravirtualization:**
- Paravirtualization requires each VM to run a frontend driver. When VMs need to access an I/O device, the VMs send I/O requests to the privileged VM (act as hypervisor) through the frontend driver, and the backend driver of the privileged VM collects the I/O request sent by each VM.
- Access to hardware drivers is transferred from the I/O frontend to the I/O backend. This mode is usually only used for hard disks and Network Interface Cards (NICs) and delivers high performance.



I/O Virtualization

- **Front-Ends and Back-Ends**
- Front-End:
 - Guest driver and emulated device
- Back-End:
 - Device implementation
 - Host physical resources



I/O Virtualization

- **Paravirtualization:**
- This reduces the performance loss of VMM and therefore delivers better I/O performance. However, **the VM OS needs to be modified** (usually Linux). Specifically, the I/O request processing method of the OS needs to be changed so that all the I/O requests can be sent to the privileged VM for processing.
- Guest uses specialized driver for optimized virtual hardware. Drivers must be implemented for each type of OS.

.



I/O Virtualization

- **IO-through (Hardware-assisted virtualization):**
- Hardware-assisted virtualization directly installs the I/O device driver in the VM OS without any change to the OS. This method is equivalent to traditional PC OS access to hardware.
- Therefore, the time required for a VM to access the I/O hardware is the same as that for a traditional PC to access the I/O hardware.



I/O Virtualization

- **IO-through (Hardware-assisted virtualization):**
- Hardware-assisted virtualization is like an intelligent information collection and processing platform. Users' requests can be directly submitted to the platform and the platform automatically processes the requests.
- Therefore, hardware-assisted virtualization outperforms full virtualization and paravirtualization in terms of I/O performance. However, hardware-assisted virtualization requires special hardware support.



I/O Virtualization

- **IO-through (Hardware-assisted virtualization):**
 - Guest can access the device directly
 - Virtually eliminates all emulation and back-end overhead
 - Each device is limited to use by one VM
 - Introduces strong coupling between Guest and hardware

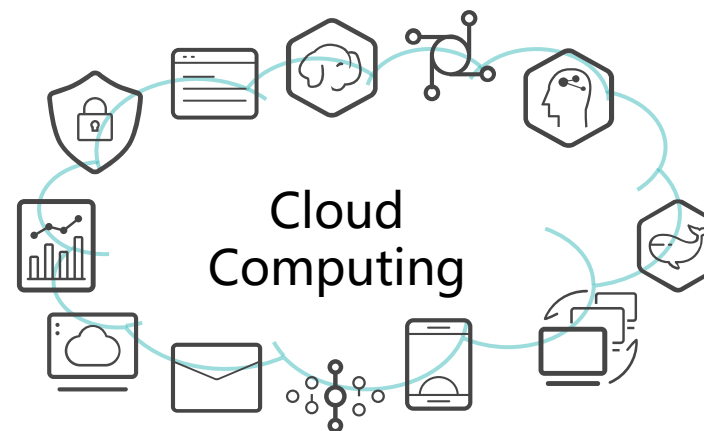


I/O Virtualization

- **I/O Virtualization:**
- **Emulation (Full Virtualization)**
 - Best option for correctness and abstraction
 - High performance cost
- **Paravirtualization**
 - Optimize driver and virtual device interaction
 - Guest is “aware” of virtualization
- **IO-through**
 - Best option for performance
 - Strong coupling with hardware



Cloud Computing and Virtualization





Cloud Computing and Virtualization

- CPU virtualization, memory virtualization, and I/O virtualization can be implemented to enable the reuse of physical resources. Multiple virtual servers can run on a physical host at the same time, and each virtual server can run different workloads. **This improves hardware utilization.**
- In **cloud computing**, we mainly **use virtualization** to implement **IaaS cloud services**. There are three cloud service models: IaaS, PaaS, and SaaS. Some PaaS and SaaS services are implemented based on virtualization



Cloud Computing and Virtualization

- Cloud computing is a **business model** that provides users with IT services anytime anywhere. Virtualization is an important technical means for cloud computing implementation.
- There are many mainstream virtualization technologies. Generally, open-source and closed-source are used for classification. Open-source technologies include KVM and Xen. Closed-source virtualization technologies include Microsoft Hyper-V, VMware vSphere, and Huawei FusionSphere.

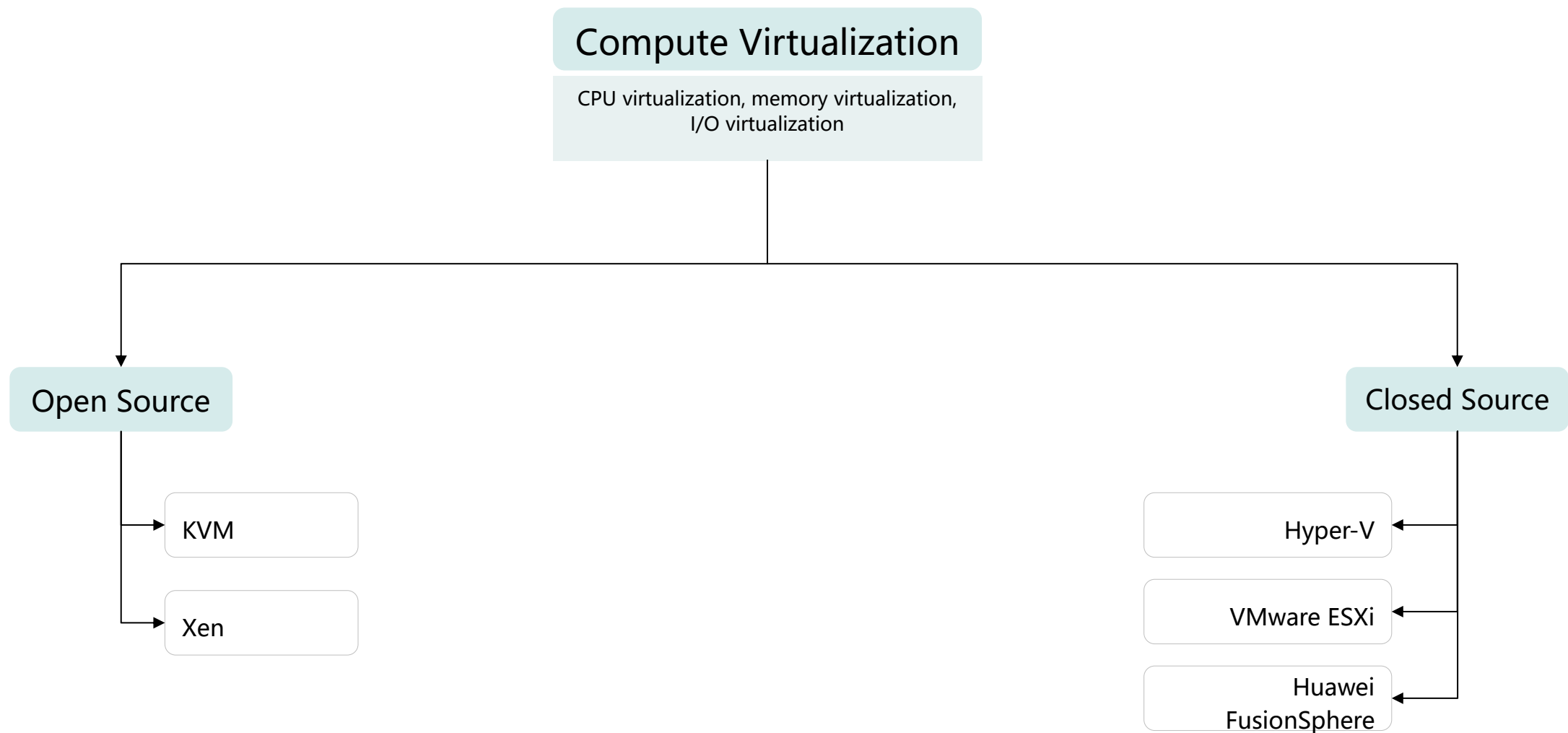


Cloud Computing and Virtualization

- **Open-source technologies** are free of charge and can be used anytime. Users can customize some special requirements based on open-source code. Once a problem occurs in the system, the system recovery strongly relies on the administrator's skillset and experience.
- **In closed-source technologies**, users cannot view or customize source code. Closed-source virtualization products are generally not free of charge and can be used out of the box. If a system problem occurs, vendors provide all-round support.



Mainstream Compute Virtualization Technologies (Hypervisors)



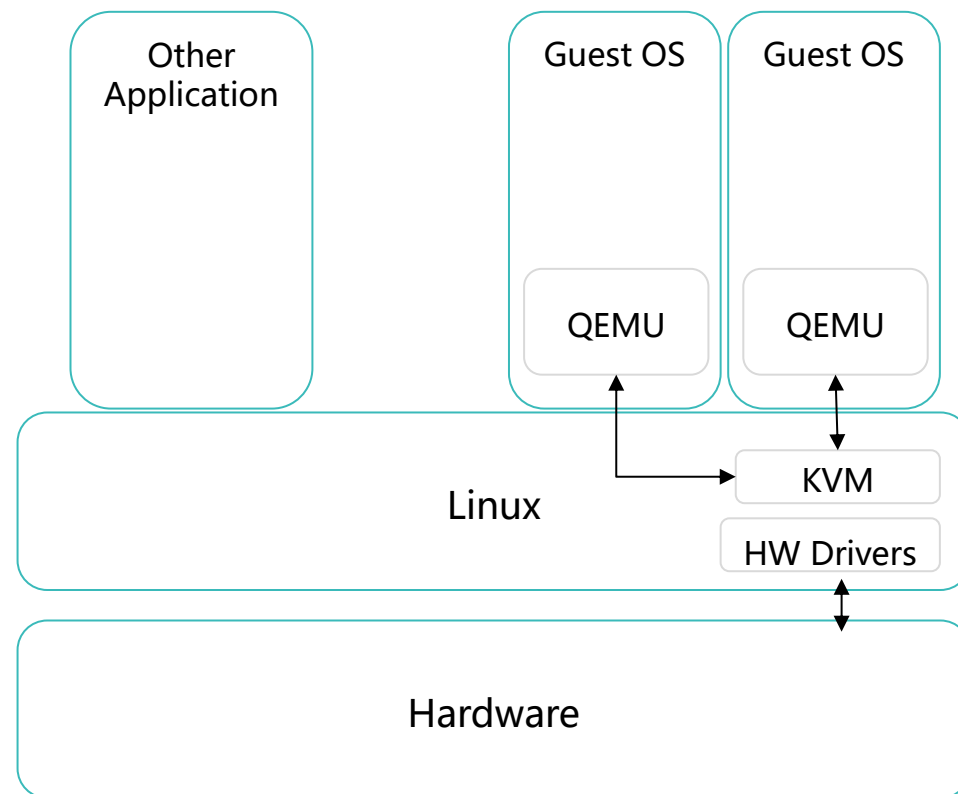


What's KVM?

- **Kernel-based Virtual Machine (KVM)** is a Type-II full virtualization solution. It is a **Linux kernel module**. A physical machine with a Linux kernel module installed can function as a hypervisor, which does not affect the other applications running on the Linux OS. Each VM is one or more processes. You can run the kill command to kill the processes.



What's KVM?



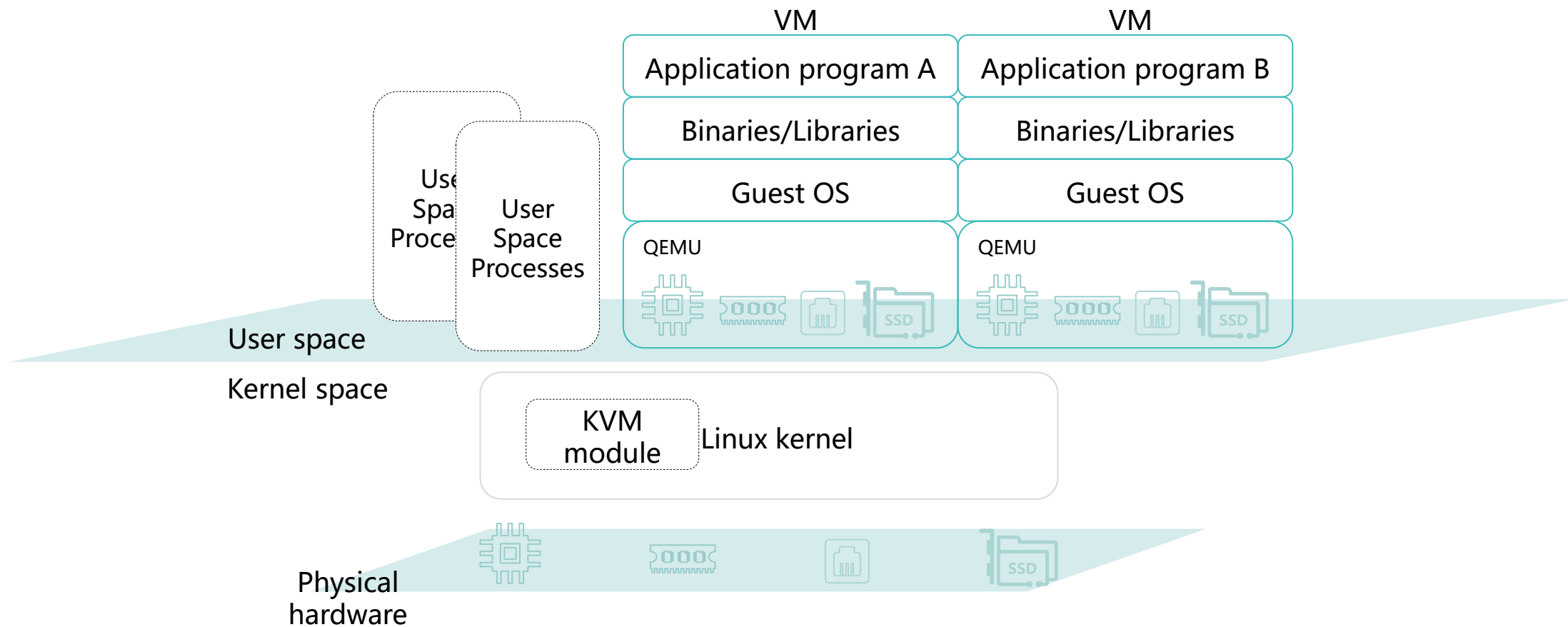


What's KVM?

- After the KVM module is installed in a common Linux OS, three running modes are added:
 - **Guest Mode:** VMs, including their CPUs, memory, and disks, run in a restricted CPU mode.
 - **User Mode:** The **quick emulator (QEMU)** typically runs in this mode. QEMU emulates I/O requests.
 - **Kernel Mode:** In this mode, the hardware can be operated. When the guest OS executes an I/O operation or privileged instruction, a request needs to be submitted to the user mode, and then the user mode initiates a hardware operation request to the kernel mode again to operate the hardware.



What's KVM?





KVM Architecture

- QEMU was not a part of KVM. It was a universal open-source virtualization emulator that uses pure software to implement virtualization. **The guest OS considers that it is interacting with hardware.**
- Actually, QEMU is interacting with hardware. This means that all interactions with the hardware need to **pass through QEMU. Therefore, the simulation performance delivered by QEMU is low.** QEMU is able to simulate CPUs and memory. **However, in KVM, only QEMU is used to simulate I/O devices.**

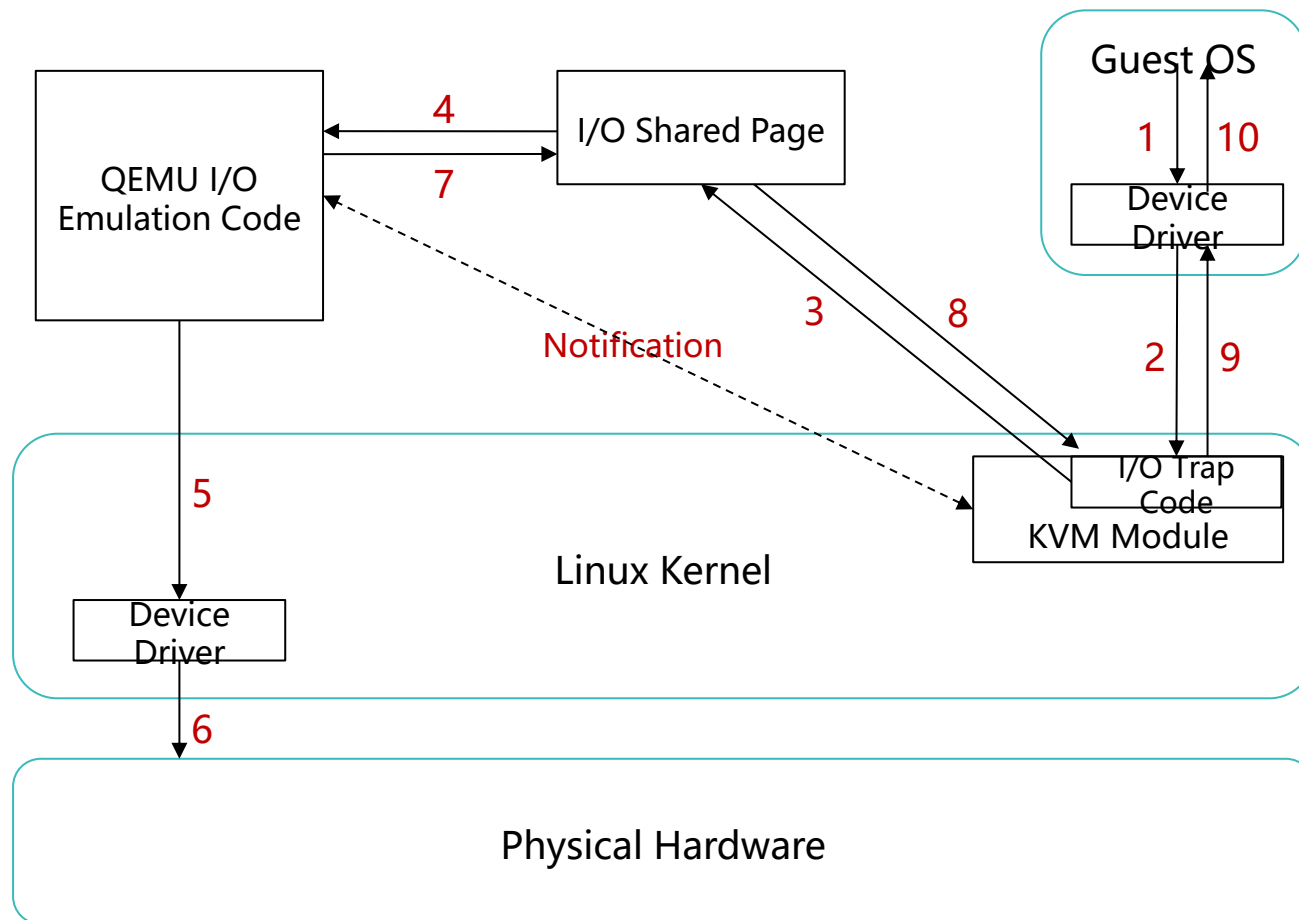


KVM Architecture

- The KVM kernel module is the core of a KVM VM. This module **initializes the CPU hardware, enables the virtualization mode, and runs the guest machine in the VM mode.**
- However, a VM requires other I/O devices such as Network Interface Cards (NICs) and hard disks besides CPUs and memory. QEMU is required to implement other virtualization functions.
- **Therefore, the KVM kernel module and QEMU form a complete virtualization technology.**



KVM I/O Process - Default





KVM I/O Process - Default

- **Let's see how VM disk operations are performed:**
 1. A disk device of a VM initiates an I/O operation request.
 2. I/O Trap Code (I/O capture program) in the KVM module captures the I/O operation request, performs corresponding processing, and then puts the processed request into the I/O shared page.
 3. The KVM module notifies QEMU that a new I/O operation request is placed in the shared page.
 4. After receiving the notification, QEMU obtains the detailed information about the I/O operation request from the shared page.



KVM I/O Process - Default

5. QEMU simulates the request and calls the device driver running in kernel mode based on the request information to perform the real I/O operation.
6. The I/O operation is then performed on physical hardware through the device driver.
7. QEMU returns the operation result to the shared page and notifies the KVM module that the I/O operation is complete.
8. I/O Trap Code reads the returned result from the shared page.
9. I/O Trap Code returns the operation result to the VM.
10. The VM returns the result to the application that initiated the operation.

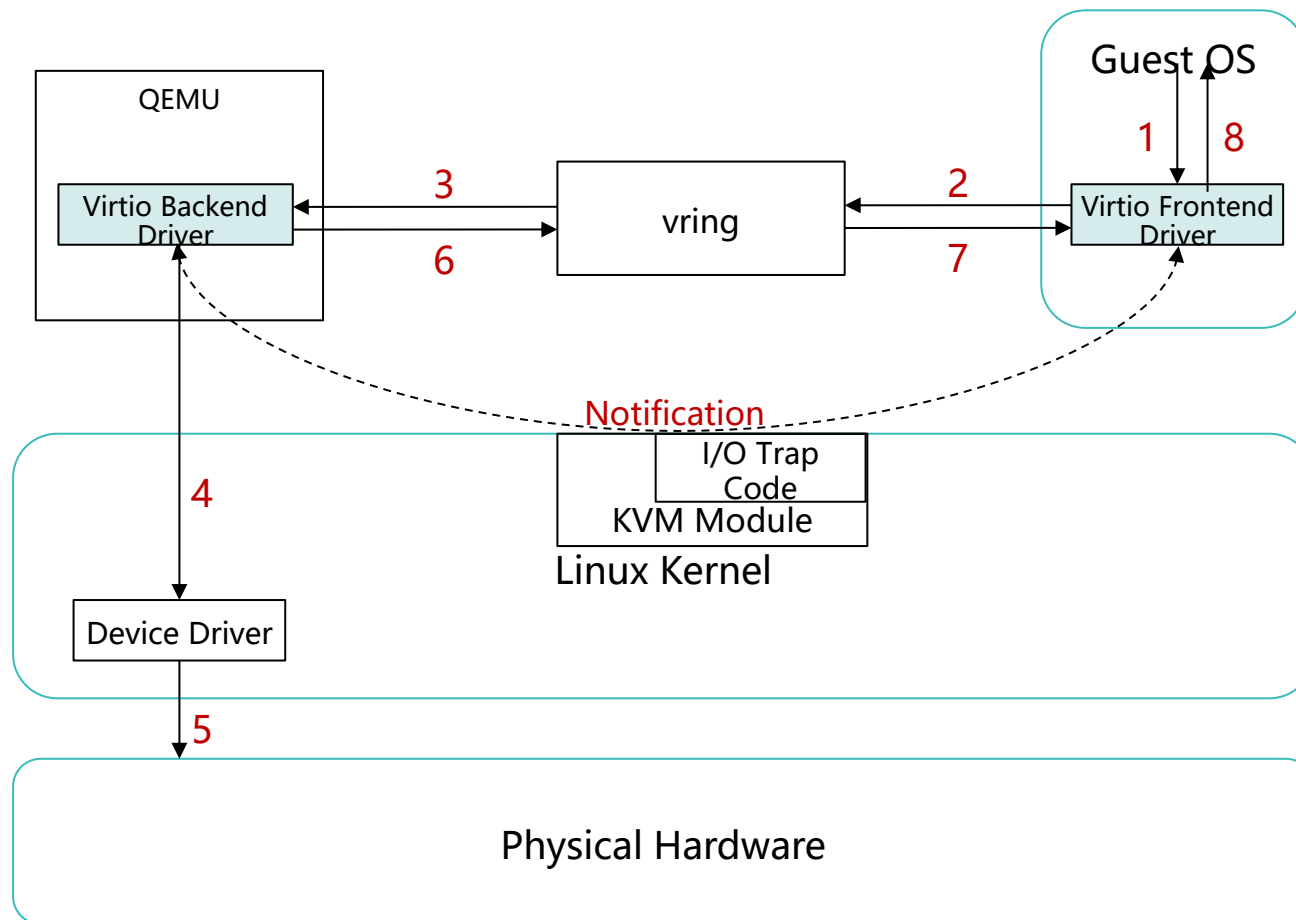


KVM I/O Process - Virtio

- In steps 2, 3, and 7 (Default) , KVM does not make any modification on the I/O operation except for capturing the request and sending the notification.
- **The Virtio** technology was developed to simplify this procedure.



KVM I/O Process - Virtio





KVM I/O Process - Virtio

- If Virtio is used, the procedure is as follows:
 1. The VM initiates an I/O operation request.
 2. The I/O operation request is not captured by the I/O capture program. Instead, the request is stored in the ring buffer between the frontend and backend drivers. At the same time, the KVM module notifies the backend driver.
 3. QEMU obtains the detailed information about the operation request from the ring buffer.



KVM I/O Process - Virtio

4. The backend driver directly calls the actual physical device driver to perform the I/O operation.
5. The operation is completed by the device driver.
6. QEMU returns the operation result to the ring buffer, and the KVM module notifies the frontend driver.
7. The frontend driver obtains the operation result from the ring buffer.
8. The frontend driver returns the result to the application that initiated the operation.

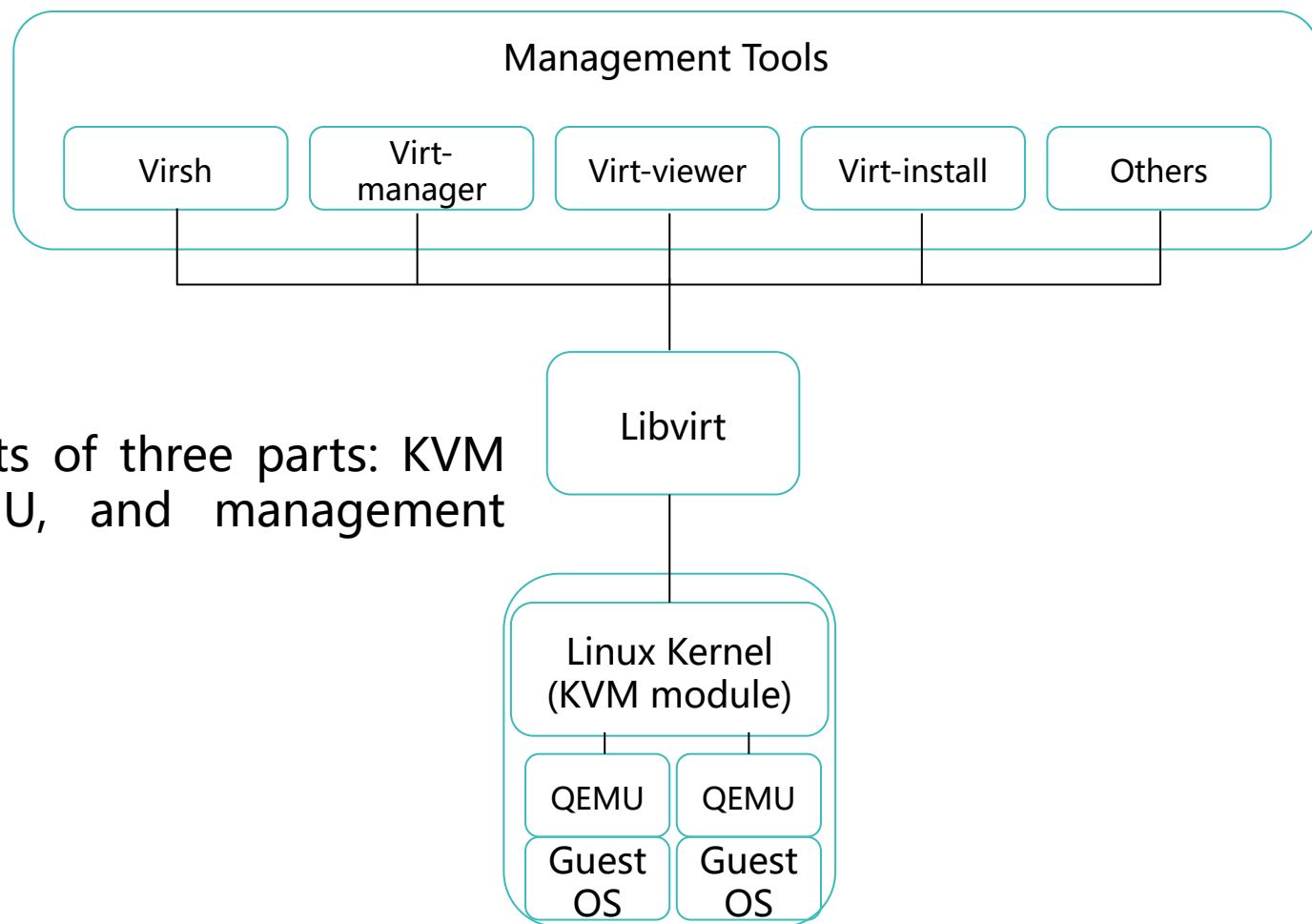


KVM I/O Process - Virtio

- **The advantages of Virtio are as follows:**
 - Saves the hardware resources required for QEMU emulation.
 - Reduces the number of I/O request paths and improves the performance of virtualization devices.
- **Virtio has some disadvantages:**
- For example, some old or uncommon devices cannot use Virtio but can only use QEMU.



KVM Architecture



A KVM system consists of three parts: KVM kernel module, QEMU, and management tool.



The Xen Hypervisor Approach

- Xen is an **open source hypervisor program** developed by Cambridge University. It creates logical pools of system resources so that many virtual machines can share the same physical resources.
- Xen is a **type 1 hypervisor (bare-metal)** that runs directly on the system hardware. Xen inserts a virtualization layer between the system hardware and the virtual machines, turning the system hardware into a pool of logical computing resources that Xen can dynamically allocate to any guest operating system.
- The operating systems running in virtual machines (Guest OS) interact with the virtual resources as if they were physical resources.



The Xen Hypervisor Approach

- **Xen** is based on **paravirtualization**, it requires that the **guest operating systems be modified** to support the Xen operating environment.
- This is probably done by **removing the performance loss** while executing the instructions requiring significant handling and by **modifying portion of the guest operating system executed by Xen**, with reference to the execution of such instructions.



The Xen Hypervisor Approach

- In paravirtualization (PV), the guest OS knows that it is running in a virtual machine and cooperates with the hypervisor to get access to the actual hardware.
- The guest OS asks the hypervisor to perform functions that would normally require direct access to hardware: e.g., accessing certain CPU registers, etc. It is much easier for the hypervisor to translate these calls instead in case of full virtualization that emulates hardware devices and network interfaces to guests.
- The result of Paravirtualization (PV) is a very lightweight and fast hypervisor.

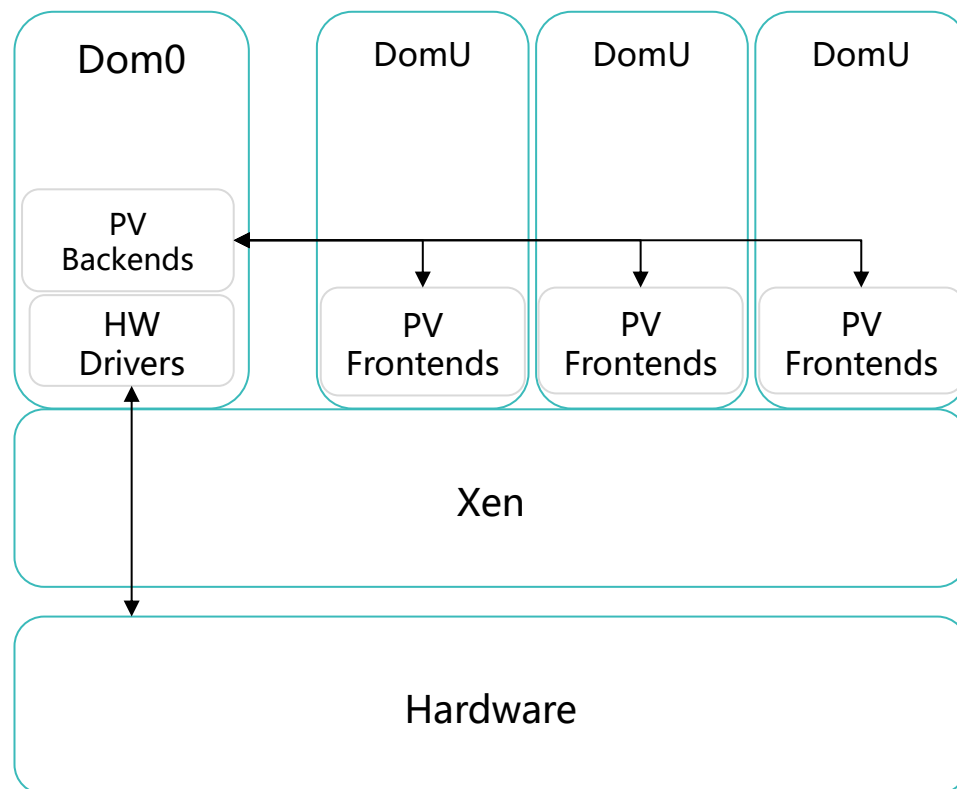


Components of the Xen hypervisor

- Xen refers to each virtual machine that runs on a system as “**domains**” .
- When Xen is first installed, it automatically creates the first domain, a **special domain** known as **Domain 0 (or dom0)**, is responsible for controlling the hypervisor and starting other guest operating systems. It is a privileged domain that can access the hardware resources and can manage all the other domains (e.g., create, destroy, save, restore, etc.)
- Other **guest operating systems** are called **DomUs** domain, where the U stands for unprivileged. This is because they cannot control the hypervisor or start/stop other domains.



The Xen Architecture





Xen Terminologies

- **Domain:** The term used by Xen to refer to a virtual machine instance.
- **Dom0:** The primary domain in a Xen system.
- **DomU:** All the other domains in a Xen system.
- **Host:** The system that provides the environment for running virtual machines.
- **Guest:** The virtual machine instance that runs on a host system.
- **PVM:** Paravirtualized Virtual machine where the Xen domains can only run modified operating systems.
- **HVM:** Hardware Virtual Machine where the Xen domains can run unmodified operating systems such as Microsoft Windows leveraging special virtualization hardware



Components of the Xen hypervisor

- **Domain 0 (Dom0):** is the management domain and is responsible for managing the system. It runs a guest operating system and is responsible for the hardware devices.
 - Dom0 is a privileged domain with direct access to the hardware. It has exclusive privileges to access the Xen hypervisor and manage all aspects of Guests.
 - Dom0 handles all access to the hardware and I/O and manages them on behalf of the users' VMs. This is because the hypervisor doesn't contain device drivers. Dom0 then shares these resources with guest operating systems.
 - The OS running on dom0 has the device drivers and performs I/O operations on behalf of unprivileged guest domains (domU);



Components of the Xen hypervisor

- **The Domain U Guest** is a modified open-sourced OS (ex: Linux, Solaris, FreeBSD or other UNIX system) that is aware of virtualization (no direct access to hardware).
- No rights to directly access hardware resources, unless especially granted.



Components of the Xen hypervisor

- To implement paravirtualization, each paravirtualized datapath consists of two parts:
 - 1) **Backend:** that lives in dom0, which provides the virtual device.
 - 2) **Frontend:** driver within the guest domain, which allows the guest OS to access the virtual device.
- The backend and frontend use a high-speed software interface based on shared memory to transfer data between the guest and dom0.

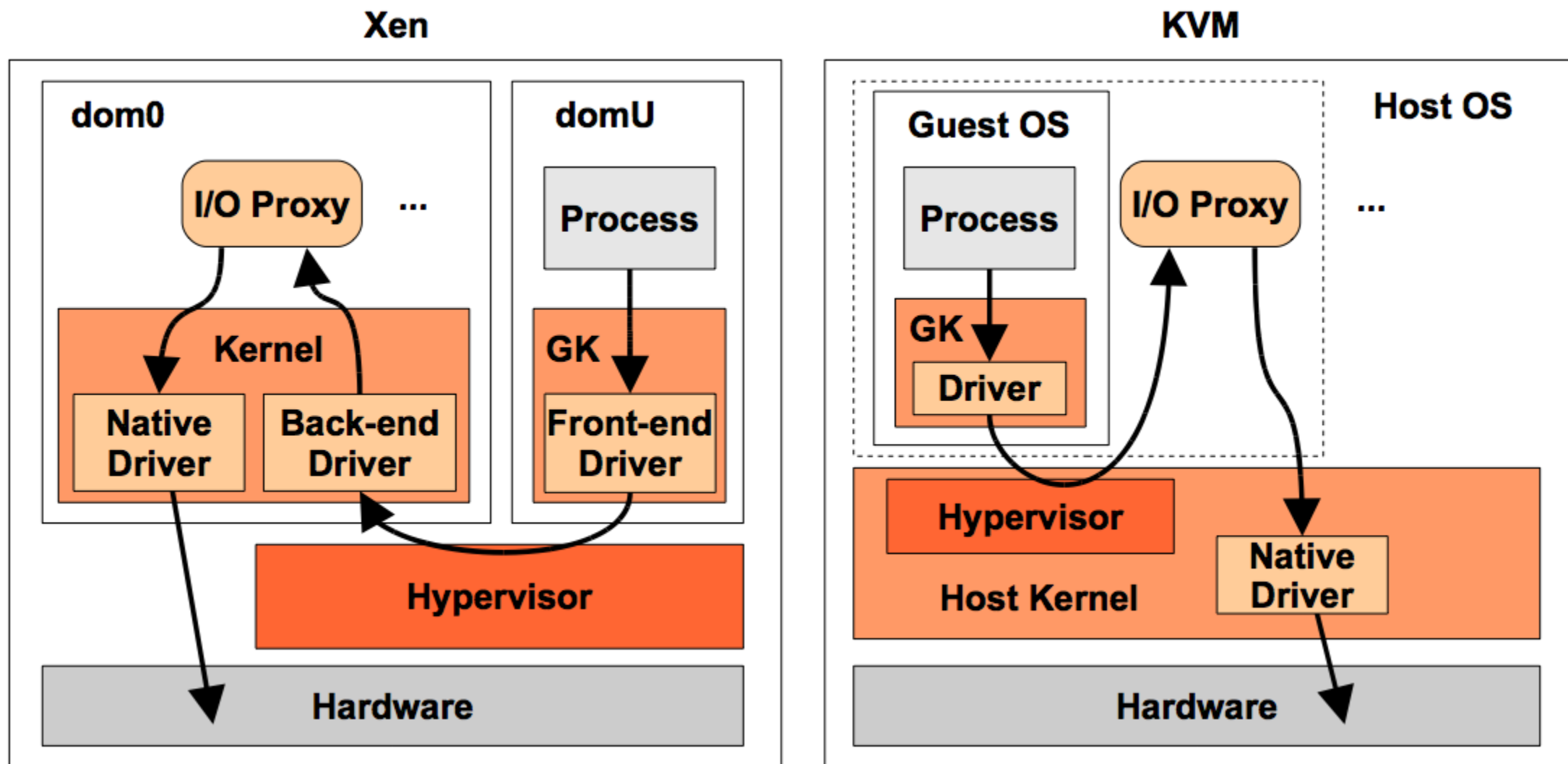


Xen vs. KVM

- In open-source virtualization technologies, Xen is on a par with KVM. KVM is full virtualization, while Xen supports paravirtualization. KVM, a module in the Linux kernel, is used to virtualize CPUs and memory. It is a process of the Linux OS.
- Different from KVM, Xen directly runs on hardware, and VMs run on Xen.
- VMs in Xen are classified as the privileged VM (Domain 0) that has the permission to directly access hardware and manage other VMs (for example, Domain U). Domain 0 must be started before other VMs. Domain U is a common VM and cannot directly access hardware resources.
- All operations on Domain U must be forwarded to Domain 0 through frontend and backend drivers. Domain 0 completes the operations and returns the results to Domain U.



Xen vs. KVM (I/O Process)





Summary

- This chapter provides basic knowledge about compute virtualization, including CPU, memory, and I/O virtualization. It introduces you to open-source virtualization technology by using KVM and Xen as an example.

The background of the image shows silhouettes of several groups of business professionals in a modern office environment. They are standing on a highly reflective floor, and their reflections are clearly visible. The entire scene is overlaid with a semi-transparent blue filter. In the center, the text "Thank You" is written in a large, white, sans-serif font, with the website address "www.huawei.com" in a smaller, white, sans-serif font directly below it.

Thank You

www.huawei.com